MetaMani: A Manipulation-Unit Framework for Compositional Long-Horizon Robot Tasks

Anonymous submission

Abstract

Long-horizon robot manipulation is often learned and executed as complete trajectories. This binds reusable operations to fixed task orders, requires new demonstrations when familiar operations are reordered, and allows local failures to disrupt subsequent execution. Existing task-level policies and planners often lack executor-independent boundaries at which completed progress can be preserved and failed operations can be retried with alternative policies. We introduce MetaMani, a framework that organizes learning and execution around reusable manipulation units. Each unit specifies a local goal, an initiation condition, and observable success and failure criteria, while independently trained policies determine how to realize the intended state change. MetaMani selects compatible policies based on reliability estimates, verifies outcomes at unit boundaries, and retries locally or switches policies without replaying completed work. This enables checkpoint reuse across reordered task compositions without joint training or additional task-level demonstrations. Across RoboCasa365, RoboTwin 2.0, and real-world tasks, MetaMani improves long-horizon completion. With a fixed low-level checkpoint, its single-policy configuration increases successful real-robot trials from 31/120 to 85/120. Heterogeneous per-MU dispatch reaches 98 of 120 (81.7%). These results demonstrate the effectiveness of organizing long-horizon manipulation around reusable and verifiable operations.

Introduction

A robot that can grasp a container, open its lid, and pour its contents should be able to recombine these operations in a different order. Trajectory-centric manipulation instead represents each composition as a complete task sequence, often requiring new task-level data or policies. The limitation is not only policy capacity or action horizon. A trajectory is asked to serve simultaneously as the unit of data collection and learning, execution, verification, and recovery. For long-horizon manipulation, this unit is too coarse. A local failure changes the physical context faced by every later action and may invalidate the remainder of the sequence. Verified progress cannot be isolated and preserved for subsequent reuse, inspection, or local recovery.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

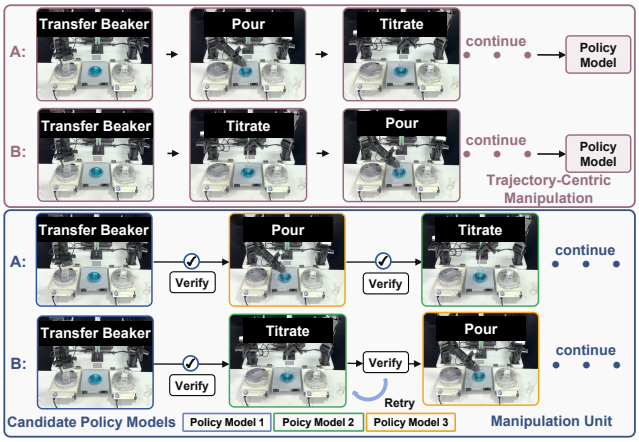


Figure 1: Trajectory-centric learning versus reusable MU composition with policy matching, verification, and recovery.

Vision-language-action (VLA) models broaden perception-conditioned action generation across tasks and embodiments (Brohan et al. 2023; Kim et al. 2024; Physical Intelligence et al. 2025; Driess et al. 2023; Jiang et al. 2023; Open X-Embodiment Collaboration 2024), while language-guided planners can decompose tasks or revise plans from feedback (Ahn et al. 2022; Huang et al. 2022; Liang et al. 2023). Yet greater policy capacity or task-level replanning does not by itself define an executor-independent operation boundary: where an operation ends, whether its intended state change was achieved, and which compatible policy should retry or take over after failure. Long-horizon competence therefore requires organizing manipulation around reusable operations whose outcomes can be verified, whose completed prefix can be preserved, and whose policy models can be exchanged locally.

In trajectory-centric manipulation, semantically distinct operations are coupled into a single data-collection, learning, and execution problem. At the data level, a new ordering or objective may require complete successful rollouts even when most constituent operations have appeared in other tasks. At the capability level, learned operations remain embedded within a task-level policy rather than exposed as units that can be selected, verified, and recombined. Knowing how

to transfer, pour, and titrate therefore does not directly provide a mechanism for executing these operations in a new long-horizon ordering (Mees et al. 2022; Jiang et al. 2023). At execution time, an early failure changes the state faced by all later operations, while the absence of an executor-independent operation boundary prevents the system from preserving completed work or retrying only the failed operation (Hu et al. 2026; Zhang and Shah 2026).

To address these challenges, we introduce manipulation units (MUs) as reusable and verifiable operations. Each MU specifies *what* local state change should be achieved through a goal, an initiation condition, and observable success and failure criteria. A policy model determines *how* to produce that change through continuous actions. Figure 1 contrasts trajectory-centric execution with MU-based composition, in which compatible policy models can be selected and outcomes verified at operation boundaries. Because the outcome contract belongs to the MU rather than a particular executor, the same unit can persist across task compositions and policy implementations. Building on this unit, we formulate MU-Structured Manipulation (MUSM), which organizes data collection, learning, execution, verification, and recovery around ordered MUs rather than complete trajectories. Consecutive MUs remain coupled through shared environment dynamics and are not assumed to be physically independent. We realize MUSM through MetaMani, which constructs an MU sequence, matches each MU to compatible models in the Frozen Capability Registry, and dispatches a model through its native adapter. Visual verification governs MU transitions. A verified success preserves the completed task prefix, while a failure triggers retry or cross-model fallback from the current state without replaying verified MUs. Our contributions are:

- We introduce manipulation units (MUs) as reusable, executor-independent semantic operations with explicit initiation and outcome contracts, and formulate MUSM to organize learning, execution, verification, and recovery around these units under shared environment dynamics.
- We introduce MetaMani, which performs capability-aware dispatch, boundary verification, and local recovery without joint policy training.
- We provide a conditional analysis of verified completion, retry budget, and error propagation, together with simulation and real-world evaluations of reuse, heterogeneous composition, and failure containment.

Related Work

Long-Horizon Policy Learning. Visuomotor manipulation policies commonly learn robot actions from expert demonstrations (Zhao et al. 2023; Chi et al. 2023). A central challenge in sequential imitation learning is compounding error: prediction errors shift future observations and may drive the policy outside its training distribution (Ross, Gordon, and Bagnell 2011). Prior work improves temporal robustness through interactive data aggregation, history conditioning, and coherent action generation (Ross, Gordon, and Bagnell 2011; Mees et al. 2022; Zhao et al. 2023; Chi et al. 2023). Another line scales multimodal robot learning across tasks, embodiments, and language instructions using large policies and heterogeneous datasets (Brohan et al. 2023; Driess et al. 2023; Jiang et al. 2023; Kim et al. 2024; Open X-Embodiment Collaboration 2024; Khazatsky et al. 2024). Recent VLA models further improve efficiency, temporal modeling, and long-horizon execution (Wang et al. 2026; Fan et al. 2025; Xie et al. 2026; Physical Intelligence et al. 2025). However, these methods generally retain task-level data collection, training, reuse, and execution management. Recomposition may therefore require additional task data, while local failures can disrupt subsequent execution. MUSM instead organizes data, training, execution, cross-task reuse, verification, and recovery around MUs.

Language-Guided Planners. Language-guided planners translate instructions into subgoals, programs, or low-level policy calls. Existing approaches ground language decisions in controller affordances (Ahn et al. 2022), incorporate observations and execution feedback (Huang et al. 2022), or express plans as executable programs and spatial objectives (Liang et al. 2023; Singh et al. 2023; Huang et al. 2023). Hierarchical VLA systems combine multimodal planners with low-level action policies for long-horizon manipulation (Hu et al. 2026). Visual feedback has also been used to revise plans, detect completion, and recover from failures (Liu et al. 2026; Zhang and Shah 2026; Lin et al. 2025). Such methods may persist subgoals, check completion, and replan, but typically bind each planning-time subgoal or controller call to one executor. MUSM instead preserves a semantic MU, together with its initiation conditions and outcome criteria, across retries, cross-model fallback, and reordered compositions.

Problem Formulation

We model the shared environment as a partially observable stochastic control system,

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, \rho_0), \tag{1}$$

where $s_t \in \mathcal{S}$ is the true physical state of the environment, which is not directly available to the robot, $o_t \in \mathcal{O}$ is the robot observation, including elapsed attempt time for verification, $a_t \in \mathcal{A}$ is a continuous robot action, $P(s_{t+1} \mid s_t, a_t)$ denotes the shared transition dynamics, and ρ_0 is the initial-state distribution. The robot receives a language instruction ℓ and must satisfy its task-level completion condition. MUSM organizes trained policy models within this shared environment without factorizing P or assuming that the state produced by one MU is irrelevant to the next. Within this shared control system, MUSM associates the task with an ordered sequence of MUs $\mathbf{u} = (u_1, \ldots, u_K)$. An MU is defined as

$$u_i = (g_i, \mathcal{S}_i, \mathcal{A}_i, \mathcal{I}_i, v_i, \Pi_i), \tag{2}$$

where g_i is a local goal, $\mathcal{S}_i \subseteq \mathcal{S}$ and $\mathcal{A}_i \subseteq \mathcal{A}$ specify the state and action components relevant to that goal, $\mathcal{I}_i$ is an initiation condition, $v_i(o_t)$ returns $\{\textsc{success}, \textsc{failure}, \textsc{continue}\}$, and Π_i is a set of compatible stochastic execution policies. Policies in Π_i may differ in architecture, training data, or control precision, provided that they can pursue g_i through an interface compatible with the MU. The evolution of u_i

remains governed by the global dynamics P over the full state, rather than by an independent MU-specific transition kernel. Each fixed pair (u_i, π_k) is representable as an option.

Boundary-Level Organization. Let m index decision boundaries, let τ_m denote the time of the mth decision boundary, and let $i_m \in \{1, \ldots, K\}$ identify the active MU at that boundary. The history h_{τ_m} records all preceding policy-model attempts, verification outcomes, and boundary observations. A failed attempt creates a new decision boundary while retaining the same active MU. A verified success advances execution to the next MU. Let $\Pi_{i_m}^{(m)} \subseteq \Pi_{i_m}$ denote the policy models that remain eligible after the preceding attempts. The active MU is admissible for execution only if

$$\mathcal{I}_{i_m}(o_{\tau_m}, \ell, h_{\tau_m}) = 1. \tag{3}$$

At an admissible decision boundary, let k_m denote the index of the policy model selected for the active MU. An organizer ω selects k_m based on the current observation, task instruction, and execution history:

$$k_m \sim \omega(\cdot \mid u_{i_m}, o_{\tau_m}, \ell, h_{\tau_m}), \qquad \pi_{k_m} \in \Pi_{i_m}^{(m)}. \tag{4}$$

The next decision boundary occurs when the verifier v_{i_m} first returns SUCCESS or FAILURE. Until that boundary, the selected policy model generates stochastic continuous controls according to

$$\begin{aligned} a_t \quad &\sim \quad \pi_{k_m}(\cdot \mid o_t, u_{i_m}), \\ &\quad\; \tau_m \le t < \tau_{m+1}. \end{aligned} \tag{5}$$

If verification succeeds and $i_m < K$, the organizer advances to the next MU by setting $i_{m+1} = i_m + 1$. Success on u_K completes the task. If verification fails, the organizer retains the active MU by setting $i_{m+1} = i_m$ and updates the next-boundary eligible set $\Pi_{i_m}^{(m+1)}$ to support policy retry or fallback from the current environment state. Execution terminates in failure when no eligible policy model remains.

The task succeeds only when every MU in the instruction reaches a verified SUCCESS boundary:

$$\mathrm{Succ}(\ell) = \mathbf{1}\left[\bigwedge_{i=1}^{K} \exists m :\; i_m = i \;\wedge\; v_i(o_{\tau_{m+1}}) = \textsc{success}\right]. \tag{6}$$

The organizer is evaluated by its complete-task success rate over task instances and stochastic rollouts induced by the initial-state distribution, shared environment dynamics, and policy models. MU-level retry and policy fallback are mechanisms for improving this task-level metric rather than separate optimization objectives.

Verification and retry analysis. We derive conditional bounds relating policy reliability, verifier errors, and retry budgets to task completion, treating them as sensitivity analyses rather than certified predictions of empirical performance. Let p_{ik} be a lower bound on the probability that policy k completes MU i from every admissible history reached under the evaluation distribution, and let ε_i^- be an upper bound on the verifier's false-rejection rate. Define $s_{ik} = p_{ik}(1 - \varepsilon_i^-)$. If policy k is attempted n_{ik} times on MU i, conditioning over attempts gives the first bound below. If this bound holds conditionally after every verified prefix, the complete-task bound follows under shared environment dynamics without assuming independence across MUs, where Q_i is the conditional MU-i verified-completion probability:

$$\begin{aligned} Q_i &\ge 1 - \prod_k (1 - s_{ik})^{n_{ik}}, \\ \Pr(\mathrm{Succ}(\ell) = 1) &\ge \prod_{i=1}^{K} Q_i. \end{aligned} \tag{7}$$

For one fixed policy k, write $s_i = s_{ik}$. With R retries after its initial attempt, the MU-level lower bound is

$$\begin{aligned} \underline{Q}_i(R) &= 1 - (1 - s_i)^{R+1}, \\ \underline{Q}_i(R+1) - \underline{Q}_i(R) &= s_i(1 - s_i)^{R+1}, \end{aligned} \tag{8}$$

which shows diminishing gain from additional retries. Finally, letting $F_i^{(1)}$ denote first-attempt failure at MU i, define $\mathrm{EPR}_i = \Pr(F_{i+1}^{(1)} \mid F_i^{(1)}, \, i+1 \text{ attempted})$. Let ε_i^+ bound false acceptance of a failed MU, and let δ_i bound first-attempt failure of MU $i+1$ after a correctly verified completion. Then

$$\mathrm{EPR}_i \le \varepsilon_i^+ + (1 - \varepsilon_i^+)\delta_i \le \varepsilon_i^+ + \delta_i. \tag{9}$$

The proof and assumptions are given in the supplementary material. The bound separates errors admitted by the verifier from residual coupling through the shared state. The registry score $\widehat{q}_{z_i k}$ estimates average success under the standardized calibration distribution. It is not treated as a certified uniform lower bound on p_{ik}.

Method

Figure 2 shows the four MetaMani components. Instruction-grounded MU records decompose a task into an ordered sequence of canonical units with success and failure criteria. The Frozen Capability Registry matches each MU to compatible policy models and records their calibrated success rates. Capability-aware dispatch selects the highest-ranked eligible model and invokes it through an adapter that preserves its native interface. Verification-gated recovery checks each outcome against the MU criteria and triggers retry or policy fallback from the current state. OpenClaw (OpenClaw 2026) uses a fixed Qwen-VL-Max model for discrete organizer decisions, while low-level control runs uninterrupted between verification points.

Instruction-Grounded MU Records

At time t, MetaMani receives the three-view RGB observation $\mathbf{I}_t = (I_t^{\mathrm{left}}, I_t^{\mathrm{front}}, I_t^{\mathrm{right}})$ from the left-wrist, front, and right-wrist cameras. These views form the visual component of o_t. Given a task instruction ℓ and initial observation $\mathbf{I}_0$, the MU constructor returns ordered implementation records:

$$\begin{aligned} F_{\mathrm{unit}}(\ell, \mathbf{I}_0) \quad &= \quad (\xi_1, \ldots, \xi_K), \\ \xi_i \quad &= \quad (z_i, \ell_i, c_i^+, c_i^-, T_i), \end{aligned} \tag{10}$$

where $z_i \in \mathcal{Z}$ identifies a canonical MU, ℓ_i specifies its local state change, c_i^+ and c_i^- define observable outcome

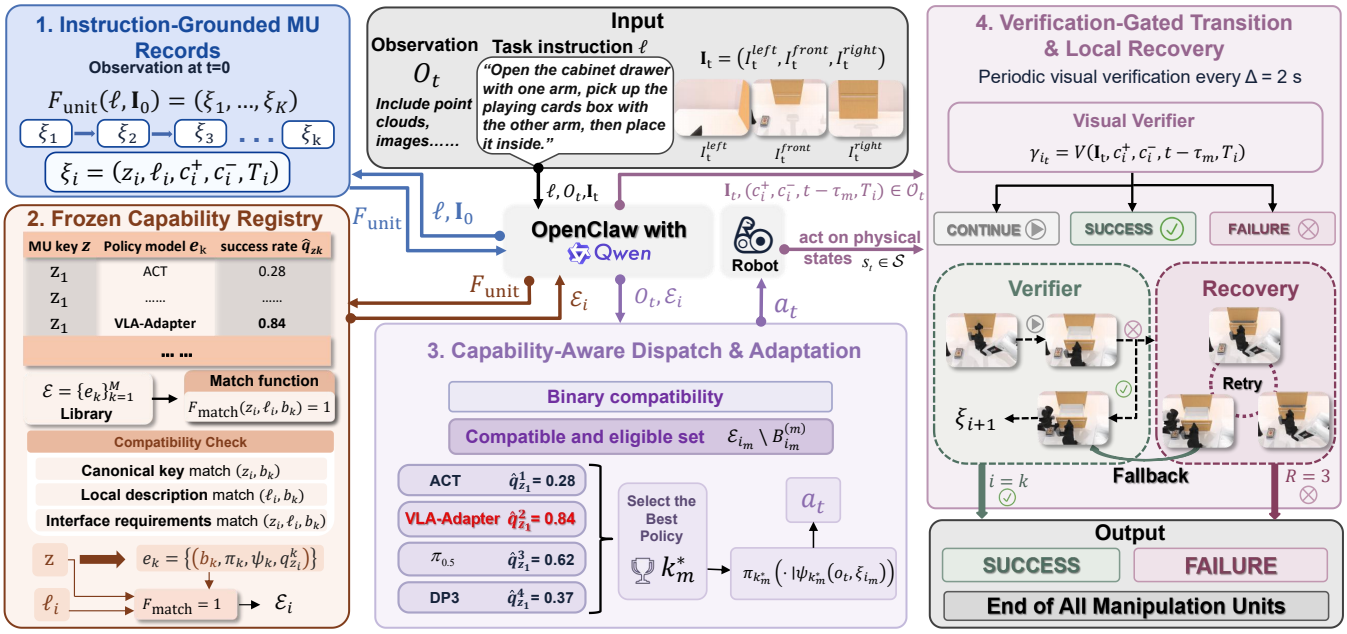


Figure 2: Overview of MetaMani. The MU constructor produces ordered MU records, while the Frozen Capability Registry matches each MU to compatible policy models. Capability-aware dispatch and adaptation then launch the most reliable model, and visual verification governs transitions and recovery.

criteria, and T_i is the timeout. The vocabulary $\mathcal{Z}$ is manually defined from recurring operations and frozen before evaluation, so the constructor cannot add new MUs online. The observation $\mathbf{I}_0$ grounds each record in the current scene, while z_i maps equivalent descriptions to a shared capability entry. Let $\bar{T}_z^{\text{demo}}$ denote the mean duration of successful demonstrations for MU z. We set $T_i = 2\bar{T}_{z_i}^{\text{demo}}$, giving each attempt twice the average demonstrated execution duration. In the MUSM formulation, ℓ_i specifies g_i, while (c_i^+, c_i^-, T_i) defines v_i. The remaining elements are instantiated by the initiation checker, visual verifier, Frozen Capability Registry, and model-specific adapters. At decision boundary τ_m, the initiation checker returns the binary decision

$$\begin{aligned} \iota_m &= F_{\text{init}}(\mathbf{I}_{\tau_m}, \ell, \xi_{i_m}, h_{\tau_m}) \\ &= \mathcal{I}_{i_m}(o_{\tau_m}, \ell, h_{\tau_m}) \in \{0, 1\}. \end{aligned} \tag{11}$$

Using the same frozen Qwen-VL-Max model with a binary output schema, F_{init} checks whether the active MU can begin. MetaMani dispatches a compatible policy model only when $\iota_m = 1$. After a failed attempt, OpenClaw reevaluates the condition from the latest three-view observation so that recovery starts from the observed post-failure state.

Frozen Capability Registry

The Frozen Capability Registry maps each MU to compatible policy models and their calibrated success rates. Let $\mathcal{E} = \{e_k\}_{k=1}^{M}$ be a library of independently trained, registered policy models, where e_k realizes the stochastic policy π_k. The registry entry for e_k stores an identifier, a capability and interface descriptor b_k, a launcher, a model-specific adapter ψ_k, and a frozen checkpoint. The descriptor b_k contains canonical MU keys, language aliases, and required observation and action fields. This common description covers policy models that use only sensor observations as well as VLAs that also receive ℓ_i.

Policy models that support the same MU may differ in reliability. MetaMani therefore calibrates each compatible MU–policy pairing before evaluation. For each pair (z, k), the registry records N_{zk} trials and S_{zk} successes and computes the offline empirical success rate as $\widehat{q}_{zk} = S_{zk}/N_{zk}$. Calibration uses privileged simulator success checks and human-corrected verifier judgments on the physical robot. Calibration trials estimate $\widehat{q}_{zk}$ but are not used to train policy models. During evaluation, the organizer accesses only the frozen rates, not the calibration observations or labels. For an MU record ξ_i, OpenClaw first filters registry entries whose sensing or action interface cannot execute the MU. Qwen-VL-Max then compares the canonical key and local description with each remaining descriptor b_k using a binary compatibility schema. The compatible policy model set is

$$\mathcal{E}_i = \{e_k \in \mathcal{E} : F_{\text{match}}(z_i, \ell_i, b_k) = 1\}. \tag{12}$$

The policies associated with $\mathcal{E}_i$ instantiate the compatible set Π_i in the MUSM formulation. The semantic comparison uses the MU description rather than exact string matching. Compatibility determines which policy models can be used, while $\widehat{q}_{zk}$ determines their ranking.

Capability-Aware Dispatch and Adaptation

The matcher and capability-aware dispatcher together instantiate the organizer ω defined in the problem formulation. At decision boundary τ_m, let $B_{i_m}^{(m)} \subseteq \mathcal{E}_{i_m}$ contain the policy models whose retry budgets for the active MU have been exhausted. The policies associated with $\mathcal{E}_{i_m} \setminus B_{i_m}^{(m)}$ instantiate

Signal	Condition	Transition
CONTINUE	any	keep current policy model
SUCCESS	$i_m < K$	record boundary and set $i_{m+1} = i_m + 1$
SUCCESS	$i_m = K$	declare task success
FAILURE	retry available	retry current policy model
FAILURE	fallback available	dispatch next policy model
FAILURE	otherwise	declare task failure

Table 1: MU transitions controlled by verification. Recovery never replays an already verified MU.

the remaining eligible set $\Pi_{i_m}^{(m)}$ in the MUSM formulation. When this set is nonempty, MetaMani selects the eligible policy model with the highest calibrated success rate:

$$k_m = \arg \max_{\pi_k \in \Pi_{i_m}^{(m)}} \widehat{q}_{z_{i_m} k}. \tag{13}$$

The dispatcher retrieves the launcher, adapter, and checkpoint from the registry entry for the selected policy model. MetaMani starts the model, and the adapter converts the current observation and MU record into the model-native input $x_t^{(k_m)}$, from which the policy samples an action:

$$x_t^{(k_m)} = \psi_{k_m}(o_t, \xi_{i_m}), \qquad a_t \sim \pi_{k_m}(\cdot \mid x_t^{(k_m)}). \tag{14}$$

Selection is therefore based on measured reliability for the active MU. The adapters retain model-native preprocessing. ACT receives RGB observations, DP3 receives RGB observations and point clouds, and $\pi_{0.5}$ and VLA-Adapter receive RGB observations together with language. For both language-conditioned policies, the adapter supplies the local MU instruction ℓ_i, rather than only the complete task instruction. On the physical platform, all adapters expose the same joint-position action space used during data collection, and the selected policy drives the robot at 25 Hz. Architecture-specific action chunking is otherwise left unchanged, requiring neither parameter sharing nor retraining.

The selected policy remains fixed during an attempt, so organizer decisions do not interrupt fast feedback control. After a verified failure, MetaMani records the policy, outcome, and boundary observation. Recovery then retries the current policy or selects the next-highest-scoring eligible alternative. This fallback handles outcomes that deviate from calibrated rates without a learned meta-policy.

Verification-Gated Transition and Local Recovery

During attempt m, the selected policy runs continuously using its native observation stream. Every $\Delta = 2$ seconds, OpenClaw sends Qwen-VL-Max the three-view RGB observation $\mathbf{I}_t$, MU record, and elapsed time:

$$\begin{aligned} v_{i_m}(o_t) &= V(\mathbf{I}_t, c_{i_m}^+, c_{i_m}^-, t - \tau_m, T_{i_m}) \\ &\in \{\text{CONTINUE}, \text{SUCCESS}, \text{FAILURE}\}. \end{aligned} \tag{15}$$

The fixed three-class schema returns SUCCESS when $c_{i_m}^+$ is visually satisfied, FAILURE when $c_{i_m}^-$ is satisfied or the elapsed time exceeds T_{i_m}, and CONTINUE otherwise. The verifier uses the same Qwen-VL-Max model and three-view RGB interface in simulation and on the physical robot. Simulator success signals are used only for offline calibration and final evaluation, never for online transitions.

OpenClaw validates every API response against its expected schema and allowed labels. Invalid responses and API failures terminate the run and are logged as infrastructure errors rather than policy failures. At each decision boundary τ_m, the active MU starts from the physical state produced by the complete execution history, including failed attempts. Retry and policy fallback continue from this state without resetting the environment. Execution therefore remains governed by the shared transition dynamics P in Eq. (1).

Table 1 summarizes the state machine. A verified SUCCESS records the boundary and advances to the next MU. A FAILURE retains the active MU and rechecks its initiation condition using the latest observation and history. Each policy receives $R = 3$ retries after its initial attempt. Once the retry budget is exhausted, the policy is added to $B_{i_m}^{(m+1)}$, and the dispatcher selects the highest-ranked remaining alternative. The task terminates when the initiation condition no longer holds or no eligible policy remains.

Recovery never replays verified MUs, and every new attempt must pass the initiation check. The observed outcome instantiates $v_{i_m}(o_{\tau_{m+1}})$ in the task-success event of Eq. (6).

Experiments

Evaluation Overview

Our experiments examine whether MU-level organization with verification and recovery improves long-horizon task completion. We further evaluate whether policy checkpoints can be reused under reordered task compositions without additional demonstrations or parameter updates, and whether heterogeneous per-MU dispatch outperforms fixing a single policy model for all MUs.

Experimental Settings

Simulation benchmarks and protocol. We evaluate MetaMani on the 50-task RoboCasa365 protocol (Nasiriany et al. 2026) and the eight RoboTwin 2.0 (Chen et al. 2025) tasks in Table 2(b), under both Easy and Hard settings. RoboCasa365 experiments use the official training data, task and scene splits, rollout count, episode horizon, and success criteria. For each RoboTwin task, all compared policies use the same 500 collected trajectories. MetaMani segments these trajectories into MUs without collecting additional demonstrations. RoboTwin evaluation likewise follows the official Easy and Hard rollout protocol. In both benchmarks, retries occur within the original rollout horizon, without resetting the environment or adding evaluation rollouts. We report the official binary task success rate (SR) for both benchmarks.

Real-world protocol. Real-world experiments use a bimanual AgileX Mobile ALOHA platform in six settings across two scenarios. Chemistry-lab tasks vary left–right titration order (BiTit-LR/RL) and pouring–titration order (Sol-PF/TF); household tasks cover cabinet retrieval and

Method	Overall ↑	Atomic-Seen ↑	Composite-Seen ↑	Composite-Unseen ↑
Diffusion Policy (Chi et al. 2023)	6.1	15.7	0.2	1.3
π_0 (Black et al. 2024)	14.8	34.6	6.1	1.1
$\pi_{0.5}$ (Physical Intelligence et al. 2025)	16.9	39.6	7.1	1.2
GR00T N1.6 (NVIDIA GEAR Team 2025b)	21.9	51.1	9.4	1.7
GR00T N1.5 (NVIDIA GEAR Team 2025a)	23.9	50.7	14.8	2.7
Qwen-RobotManip (Yuan et al. 2026)	35.9	68.6	20.1	14.9
RLDX-1 (Kim et al. 2026)	36.0	67.6	27.9	8.5
ABot-M0.5 (Chen et al. 2026)	40.3	75.6	37.7	3.3
MetaMani-$\pi_{0.5}$	28.1	55.2	19.8	6.1
MetaMani-RLDX-1	48.0	78.9	44.3	16.8
MetaMani (ours)	**55.6**	**85.1**	**52.7**	**25.2**

Method	Dump Bin BigBin		Blocks Ranking RGB		Lift Pot		Place Burger Fries		Place Can Basket		Place Dual Shoes		Put Object Cabinet		Stack Bowls Two		Mean	
	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard	Easy	Hard
ACT (Zhao et al. 2023)	87	0	91	0	**100**	0	97	0	77	0	84	0	85	0	96	0	89.6	0.0
DP3 (Ze et al. 2024)	93	75	70	30	**100**	0	96	15	83	2	26	0	80	9	**100**	5	81.0	17.0
$\pi_{0.5}$ (Physical Intelligence et al. 2025)	89	59	72	6	**100**	42	98	50	80	5	51	6	89	4	**100**	51	84.9	27.9
VLA-A (Wang et al. 2026)	95	39	0	0	80	0	89	1	8	0	39	1	48	0	93	0	56.5	5.1
MetaMani-ACT	92	35	**95**	15	**100**	40	**99**	15	**85**	**10**	**90**	**10**	90	**25**	98	50	93.6	25.0
MetaMani-DP3	96	**82**	87	**45**	**100**	12	98	22	**85**	7	32	5	87	15	**100**	12	85.6	25.0
MetaMani-$\pi_{0.5}$	93	62	78	12	**100**	45	**99**	52	84	9	58	**10**	**91**	10	**100**	54	87.9	31.8
MetaMani-VLA-A	**97**	44	5	2	85	5	93	5	12	2	45	5	55	5	96	5	61.0	9.1
MetaMani (ours)	**97**	**82**	**95**	**45**	**100**	**48**	**99**	**55**	**85**	**10**	**90**	**10**	**91**	**25**	**100**	**55**	**94.6**	**41.3**

Table 2: Simulation task success rate (%). (a) RoboCasa365 multi-task benchmark results. (b) RoboTwin 2.0 tasks under Easy and Hard settings. VLA-A denotes VLA-Adapter. MetaMani denotes the full heterogeneous system, selecting for each MU one policy from those used by the single-policy MetaMani variants listed above. Higher is better.

placement (Cab-Take/Place). Figure 3(a) illustrates the real-world MUs, denoted T_A–T_H. No complete task trajectory is used to train MU policies. Within each task setting, all methods are evaluated on the same 20 matched initial conditions. Capability scores are calibrated using a disjoint set of $N_{\text{cal}} = 20$ trials for each compatible MU–policy pair, and both the scores and compatibility masks are frozen before evaluation. Real-world metrics are SR and EPR as defined above, normalized progress (Prog; verified-MU fraction), and failure containment rate (FCR; first-attempt-failure recovery without verified-prefix replay). Supplementary material gives full protocols, calibration, statistics, robustness, and data budgets.

Simulation Results

RoboCasa365. RoboCasa365 tests whether learned operations remain effective in longer and unseen compositions. With the RLDX-1 backbone fixed, MetaMani raises overall SR from 36.0% to 48.0%, Composite-Seen from 27.9% to 44.3%, and Composite-Unseen from 8.5% to 16.8%. The gain increases from 11.3 points on Atomic-Seen to 16.4 points on Composite-Seen, while success on Composite-Unseen nearly doubles. Thus, MU organization recovers complete tasks that the unchanged low-level backbone rarely finishes as trajectories, with the largest absolute benefit on multi-operation compositions. MetaMani further reaches 55.6% overall and 25.2% on Composite-Unseen, exceeding the best reported baseline results by 15.3 and 10.3 points, respectively. Its 85.1% Atomic-Seen SR shows that heterogeneous dispatch retains strong individual-operation competence while extending coverage to harder compositions beyond any single policy.

RoboTwin 2.0. RoboTwin Hard exposes a sharp execution gap. ACT scores 0.0% across all eight tasks, whereas MetaMani-ACT raises the same backbone to 25.0% mean SR, reaching 50% on Stack Bowls Two, 40% on Lift Pot, and 35% on Dump Bin BigBin. Per-MU selection among complementary policies further raises MetaMani to 41.3%, with nonzero success on every Hard task and performance matching or exceeding the best fixed-policy variant task by task. MetaMani also reaches 94.6% on Easy. These results show that verified MU execution converts fixed low-level competence into task completion, while heterogeneous dispatch expands coverage beyond any single policy.

Real-World Results

MU Organization versus Trajectory Coupling. To control policy capacity, this experiment uses $\pi_{0.5}$ as the sole policy model. TRAJECTORY-E2E trains a task-level policy from complete trajectories under a matched interaction-frame budget. FLAT-SINGLE and METAMANI-SINGLE share the same MU-trained checkpoint, but only the latter enables boundary

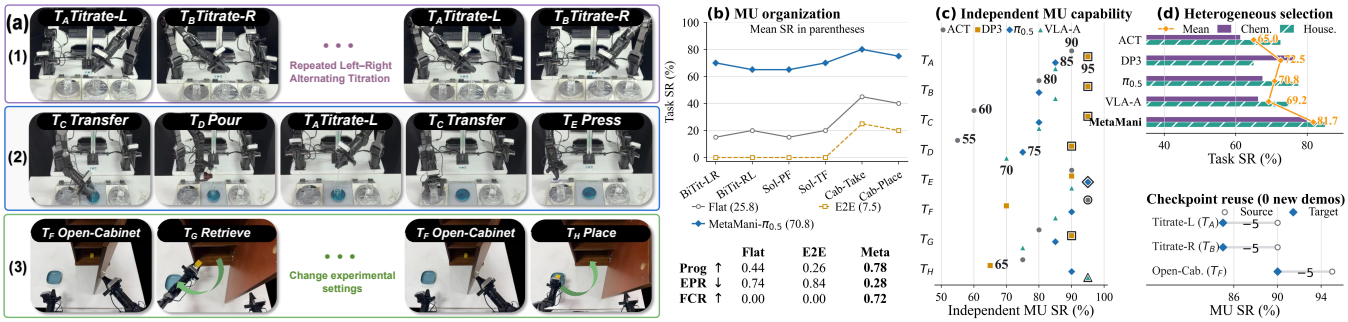


Figure 3: Real-world evaluation. (a) Representative real-world MU sequences. (b) Comparison of MU-organized, flat, and trajectory-coupled execution. (c) Independent MU success rates characterize the four policy models. (d) Heterogeneous versus single-policy execution and checkpoint reuse under reordered task compositions. VLA-A denotes VLA-Adapter.

Variant	Same-policy retry	Policy fallback	Max attempts/MU	Task SR ↑	Prog ↑	EPR ↓	FCR ↑	Attempts/MU ↓	Rel. time ↓
Verified-Stop	No	No	1	38.3%	51.7%	N/A	0.0%	1.00	1.00×
Retry-Only	Yes	No	$R+1$	68.3%	79.2%	35.2%	60.4%	1.82	1.38×
Fallback-Only	No	Yes	$\lvert\Pi_i\rvert \le 4$	57.5%	68.3%	48.7%	43.8%	1.47	1.21×
Retry + Fallback	Yes	Yes	$(R+1)\lvert\Pi_i\rvert \le 16$	81.7%	90.8%	12.6%	87.3%	1.38	1.52×

Table 3: Recovery ablation on six real-world settings (120 matched trials/variant). Attempts/MU counts invocations per attempted MU position; relative time is normalized to VERIFIED-STOP. Attempt caps vary by mechanism.

verification and local recovery with $R = 3$ retries.

Figure 3(b) compares MU-level, flat, and trajectory-coupled execution. Across 120 trials each, TRAJECTORY-E2E completes 9 (7.5%), whereas FLAT-SINGLE completes 31 (25.8%). Given matched interaction-frame budgets, this gap reflects the benefit of MU-level training over complete-trajectory training. Using the same checkpoint as FLAT-SINGLE, METAMANI-SINGLE completes 85 trials (70.8%), a 45.0-point gain, and reaches 65–80% SR across all settings. It raises Prog from 44% to 78%, reduces EPR from 74% to 28%, and increases FCR from 0% to 72%. These gains come from verified transitions and local recovery with the same checkpoint. Table 3 compares retry, fallback, and their combination under mechanism-specific caps using the same four-policy pool. VERIFIED-STOP reaches 38.3% task SR and has undefined EPR, stopping at the first failed MU. RETRY-ONLY and FALLBACK-ONLY reach 68.3% and 57.5%, respectively, while their combination reaches 81.7% task SR and 87.3% FCR. Although RETRY + FALLBACK permits up to 16 attempts per MU, it averages 1.38 attempts and 1.52× the execution time of VERIFIED-STOP. Recovery thus incurs moderate execution-time overhead while yielding substantial gains in task completion and failure containment.

Policy Composition and Checkpoint Reuse. Figure 3(c) reports the independent MU success rates used to calibrate the Frozen Capability Registry. The four policy models exhibit complementary strengths, each achieving the highest success rate on at least one operation. We isolate heterogeneous dispatch by keeping the MU constructor, verifier, retry budget, and MU data fixed. MetaMani selects $\arg\max_k \widehat{q}_{z_i k}$ among ACT, DP3, $\pi_{0.5}$, and VLA-Adapter, whereas each single-policy variant fixes one model across all MUs. DP3 is strongest on chemistry tasks at 76.25%, whereas $\pi_{0.5}$ is strongest on household tasks at 77.5%. MetaMani reaches 80.0%/85.0% and an overall 81.7%, compared with 72.5% for the strongest single-policy variant. Per-MU dispatch thus composes the strengths of different policies rather than selecting one globally dominant model.

Figure 3(d, bottom) evaluates MU-level checkpoint reuse under reordered operation sequences. Each 5-percentage-point source–target difference corresponds to one of 20 trials. Using the same checkpoints, MetaMani completes 27 of 40 trials (67.5%) in the reordered target compositions, matching the aggregate 27 of 40 in the source compositions. This parity is achieved without target-task demonstrations or replacing any MU checkpoint. It indicates that learned MU capabilities are associated with individual operations rather than fixed task orders, allowing familiar operations to be recomposed without training a new task-level policy. Together, Figure 3(d) shows checkpoint reuse across compositions and reliable per-MU selection.

Conclusion

We formulate MUSM around reusable MUs with explicit outcome contracts rather than complete trajectories. MetaMani combines boundary verification, local recovery, and heterogeneous dispatch, improving compositional success in simulation and real-world experiments. It preserves verified progress and enables failed MUs to be retried without replaying completed operations. Analysis covers verifier error and retry budget. Limitations are a fixed MU vocabulary, a calibrated capability registry, and visually verifiable outcomes.

References

Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691.

Black, K.; Brown, N.; Driess, D.; Esmail, A.; Equi, M.; Finn, C.; Fusai, N.; Groom, L.; Hausman, K.; Ichter, B.; et al. 2024. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*.

Brohan, A.; Brown, N.; Carbajal, J.; Chebotar, Y.; Chen, X.; Choromanski, K.; Ding, T.; Driess, D.; Dubey, A.; Finn, C.; et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818.

Chen, R.; Yang, Y.; Tang, Z.; Huo, D.; Lin, T.; Wu, H.; Liu, H.; Chen, Y.; Zheng, L.; Yuan, B.; Li, T.; Wang, M.; Qi, D.; Hu, B.; Mei, W.; Xuan, Y.; Yang, H.; Zhu, Y.; Xu, M.; Ma, Z.; and Chang, X. 2026. ABot-M0.5: Unified Mobility-and-Manipulation World Action Model. *arXiv preprint arXiv:2607.00678*.

Chen, T.; Chen, Z.; Chen, B.; Cai, Z.; Liu, Y.; Li, Z.; Liang, Q.; Lin, X.; Ge, Y.; Gu, Z.; et al. 2025. RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation. arXiv:2506.18088.

Chi, C.; Feng, S.; Du, Y.; Xu, Z.; Cousineau, E.; Burchfiel, B. C. M.; and Song, S. 2023. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*. Daegu, Republic of Korea.

Driess, D.; Xia, F.; Sajjadi, M. S. M.; et al. 2023. PaLM-E: An Embodied Multimodal Language Model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 8469–8488.

Fan, Y.; Bai, S.; Tong, X.; Ding, P.; Zhu, Y.; Lu, H.; Dai, F.; Zhao, W.; Liu, Y.; Huang, S.; Fan, Z.; Chen, B.; and Wang, D. 2025. Long-VLA: Unleashing Long-Horizon Capability of Vision Language Action Model for Robot Manipulation. In Lim, J.; Song, S.; and Park, H.-W., eds., *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, 2018–2037. PMLR.

Hu, J.; Shridhar, M.; Lu, C.; Shah, D.; Chiang, H.-T. L.; Tan, J.; and Xie, A. 2026. What Matters in Orchestrating Robot Policies: A Systematic Study of Hierarchical VLA Agents. arXiv:2606.10267.

Huang, W.; Wang, C.; Zhang, R.; Li, Y.; Wu, J.; and Fei-Fei, L. 2023. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. In Tan, J.; Toussaint, M.; and Darvish, K., eds., *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, 540–562. PMLR.

Huang, W.; Xia, F.; Xiao, T.; Chan, H.; Liang, J.; Florence, P.; Zeng, A.; Tompson, J.; Mordatch, I.; Chebotar, Y.; Sermanet, P.; Brown, N.; Jackson, T.; Luu, L.; Levine, S.; Hausman, K.; and Ichter, B. 2022. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *Proceedings of the Conference on Robot Learning*, volume 205, 1769–1782.

Jiang, Y.; Gupta, A.; Zhang, Z.; Wang, G.; Dou, Y.; Chen, Y.; Fei-Fei, L.; Anandkumar, A.; Zhu, Y.; and Fan, L. 2023. VIMA: Robot Manipulation with Multimodal Prompts. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 14975–15022.

Khazatsky, A.; Pertsch, K.; Nair, S.; Balakrishna, A.; et al. 2024. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. In *Proceedings of Robotics: Science and Systems*.

Kim, D.; Jang, H.; Koo, M.; Jang, S.; Kim, T.; Kim, B.; Yoon, B.; et al. 2026. RLDX-1 Technical Report. *arXiv preprint arXiv:2605.03269*.

Kim, M. J.; Pertsch, K.; Karamcheti, S.; Xiao, T.; Balakrishna, A.; Nair, S.; Rafailov, R.; Foster, E.; Lam, G.; Sanketi, P.; et al. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. arXiv:2406.09246.

Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; and Zeng, A. 2023. Code as Policies: Language Model Programs for Embodied Control. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 9493–9500.

Lin, Z.; Duan, J.; Fang, H.; Fox, D.; Krishna, R.; Tan, C.; and Wen, B. 2025. FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models. *arXiv preprint arXiv:2510.01642*.

Liu, Z.; Ning, X.; Hu, Z.; Xie, X.; Li, W.; Tang, Z.; Wang, C.; Yang, Z.; Wang, H.; Liu, Y.; and Pu, Z. 2026. Goal2Skill: Long-Horizon Manipulation with Adaptive Planning and Reflection. *arXiv preprint arXiv:2604.13942*.

Mees, O.; Hermann, L.; Rosete-Beas, E.; and Burgard, W. 2022. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *IEEE Robotics and Automation Letters*, 7(3): 7327–7334.

Nasiriany, S.; Nasiriany, S.; Maddukuri, A.; and Zhu, Y. 2026. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. In *International Conference on Learning Representations*.

NVIDIA GEAR Team. 2025a. GR00T N1.5: An Improved Open Foundation Model for Generalist Humanoid Robots. NVIDIA Research.

NVIDIA GEAR Team. 2025b. GR00T N1.6: An Improved Open Foundation Model for Generalist Humanoid Robots. NVIDIA Research.

Open X-Embodiment Collaboration. 2024. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. In *Proceedings of the IEEE International Conference on Robotics and Automation*.

OpenClaw. 2026. OpenClaw. https://github.com/openclaw/openclaw. Open-source personal AI assistant, version 2026.3.8, accessed 2026-03-09.

Physical Intelligence; Black, K.; Brown, N.; Darpinian, J.; Dhabalia, K.; Driess, D.; Esmail, A.; Equi, M.; Finn, C.; Fusai, N.; et al. 2025. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. arXiv:2504.16054.

Ross, S.; Gordon, G.; and Bagnell, D. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In Gordon, G.; Dunson, D.; and Dudík, M., eds., *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, 627–635. Fort Lauderdale, FL, USA: PMLR.

Singh, I.; Blukis, V.; Mousavian, A.; Goyal, A.; Xu, D.; Tremblay, J.; Fox, D.; Thomason, J.; and Garg, A. 2023. ProgPrompt: Generating Situated Robot Task Plans Using Large Language Models. In *2023 IEEE International Conference on Robotics and Automation*, 11523–11530. IEEE.

Wang, Y.; Ding, P.; Li, L.; Cui, C.; Ge, Z.; Tong, X.; Song, W.; Zhao, H.; Zhao, W.; Hou, P.; Huang, S.; Tang, Y.; Wang, W.; Zhang, R.; Liu, J.; and Wang, D. 2026. VLA-Adapter: An Effective Paradigm for Tiny-Scale Vision-Language-Action Model. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(22): 18638–18646.

Xie, Z.; Han, Z.; Wei, X.; Sun, S.; Li, Y.; and Zhao, J. 2026. S^2-VLA: State-Space Guided Vision-Language-Action Models for Long-Horizon Manipulation. *arXiv preprint arXiv:2606.27872*. Accepted to IJCAI 2026.

Yuan, H.; Liang, Z.; Chen, A.; Wang, Y.; Li, H.; Lin, P.; Huang, Y.; Lei, Z.; Zhang, T.; Zhang, J.; Zhang, J.; Fan, J.; Zhou, G.; Peng, Q.; Lv, C.; Chen, X.; Yang, A.; Huang, F.; Lin, J.; Liu, D.; Zhou, J.; Wu, C.; and Chen, X.-H. 2026. Qwen-RobotManip Technical Report: Alignment Unlocks Scale for Robotic Manipulation Foundation Models. *arXiv preprint arXiv:2606.17846*.

Ze, Y.; Zhang, G.; Zhang, K.; Hu, C.; Wang, M.; and Xu, H. 2024. 3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations. In *Proceedings of Robotics: Science and Systems*. Delft, Netherlands.

Zhang, M.; and Shah, D. 2026. Visual Verification Enables Inference-Time Steering and Autonomous Policy Improvement. In *Proceedings of Robotics: Science and Systems*.

Zhao, T. Z.; Kumar, V.; Levine, S.; and Finn, C. 2023. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems*. Daegu, Republic of Korea.